Hands-On Activity 7 | Purposeful Use of Color

Name : Duenas Ralph Florenz F.
Course Code and Title : CPE 031 VDA
Date Submitted : 9/14/2026
Instructor : Engr. Jimlord Quejado

---

1. Objectives:

This activity aims to develop students' ability to purposefully use color in data visualization by applying color perception, color schemes, custom HSL colors, and avoiding common mistakes.

2. Intended Learning Outcomes (ILOs):

By the end of this module the students should be able to demonstrate:

1. Analyze data visualizations to identify how hue, saturation, luminance, and color schemes affect perception and interpretation.
2. Evaluate charts for effective color use including contrast, consistency, accessibility (colorblind-safe), and data-ink efficiency and recommend specific improvements.
3. Create clear, accurate, and accessible visualizations by applying appropriate color schemes, custom HSL palettes, and preattentive/Gestalt principles.

3. Discussions:

Color is one of the most powerful tools in data visualization, but it must be applied with purpose to enhance understanding rather than distract the audience. The effective use of color goes beyond aesthetics; it helps direct attention, establish categories, and reveal patterns that may otherwise be overlooked.

-Enhancing Clarity: Colors should be chosen to make the data more readable,

not overwhelming. Overusing color can confuse the viewer, while thoughtful use highlights what is most important.

-Categorical Distinction: Distinct colors help separate groups or categories, making comparisons easier and relationships clearer. For example, different product lines or regions can be distinguished with unique colors.

-Highlighting and Emphasis: A single, contrasting color can be used to emphasize a particular data point or trend, drawing the audience's focus to critical information.

-Sequential and Diverging Scales: When representing numerical or continuous data, gradient color schemes (light to dark) indicate magnitude, while diverging scales (two contrasting colors meeting at a midpoint) highlight deviations from a norm or average.

-Accessibility Considerations: Purposeful color use also accounts for color vision deficiencies. Selecting color palettes that remain distinguishable to all viewers ensures inclusivity and accurate interpretation.

-Consistency and Professionalism: Reusing the same colors across multiple charts and dashboards helps maintain coherence, avoids misinterpretation, and conveys professionalism.

In short, purposeful use of color in data visualization ensures that design decisions support the primary goal: to communicate insights clearly, accurately, and effectively.

4. Procedures:

Color and Perception

1. Color and Perception

Human perception of color is not purely objective—it depends on context, contrast, and how our eyes and brain interpret visual signals. For example, the same shade of gray can appear lighter or darker depending on the surrounding background. This means that in visualization, color must be applied carefully, because perception can influence how data is understood.

1. Attributes of Color: Hue, Saturation, and Luminance

Hue: The type of color itself (red, blue, green, etc.).

Saturation: The intensity or purity of the color (a vivid red vs. a faded pink).

Luminance (Brightness/Value): The lightness or darkness of a color (light blue vs. dark blue). These three attributes together determine how a color is seen and how effectively it communicates meaning in visualization.

1. Color Psychology and Color Symbolism

Colors carry psychological associations and cultural meanings:

Red → urgency, warning, or importance.

Green → growth, safety, or approval.

Blue → calm, trust, or professionalism.

Yellow → caution or attention. While useful, designers should be cautious, since meanings can vary by culture.

1. Perceived Color

The perception of color depends on context and surroundings. For example, a mid-gray square will look lighter on a dark background and darker on a light background. This phenomenon illustrates how perceived color is not absolute, but relative.

1. Movement

Movement is a strong but less commonly used preattentive attribute (especially in static charts). Our brains are highly sensitive to motion—it immediately captures attention.

- Flicker/Animation: Blinking elements or pulsing dots draw the eye faster than color or size changes.
- Motion Path: Direction of movement can show flow or change over time.
- Dynamic Visualization: Used more in digital dashboards and interactive graphics rather than static charts.

Coding Procedure

In [ ]:
```
#Import libraries
import matplotlib.pyplot as plt
```

```
import numpy as np
import colorsys
```

In [ ]:

```
# Function to convert HSL (Hue, Saturation, Luminance) to RGB
def hsl_to_rgb(h, s, l):
    return colorsys.hls_to_rgb(h, l, s)

# Create figure and axes
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# ----------------------------
# 1. Hue demonstration
# ----------------------------
hues = np.linspace(0, 1, 6, endpoint=False)  # 6 different hues
colors_hue = [hsl_to_rgb(h, 1, 0.5) for h in hues]

axes[0, 0].bar(range(len(hues)), [1]*len(hues), color=colors_hue)
axes[0, 0].set_title("Hue (types of color)")
axes[0, 0].axis("off")

# ----------------------------
# 2. Saturation demonstration
# ----------------------------
saturations = np.linspace(0.2, 1, 5)  # 5 levels of saturation
colors_sat = [hsl_to_rgb(0, s, 0.5) for s in saturations]  # fixed hue (r

axes[0, 1].bar(range(len(saturations)), [1]*len(saturations), color=color
axes[0, 1].set_title("Saturation (color intensity)")
axes[0, 1].axis("off")

# ----------------------------
# 3. Luminance demonstration
# ----------------------------
luminances = np.linspace(0.2, 0.8, 5)  # 5 brightness levels
colors_lum = [hsl_to_rgb(0.6, 1, l) for l in luminances]  # fixed hue (bl

axes[1, 0].bar(range(len(luminances)), [1]*len(luminances), color=colors_
axes[1, 0].set_title("Luminance (light vs dark)")
axes[1, 0].axis("off")

# ----------------------------
# 4. Perceived Color demonstration
# ----------------------------
# Left: gray on dark background, Right: gray on light background
axes[1, 1].set_xlim(0, 1)
axes[1, 1].set_ylim(0, 1)
axes[1, 1].axis("off")

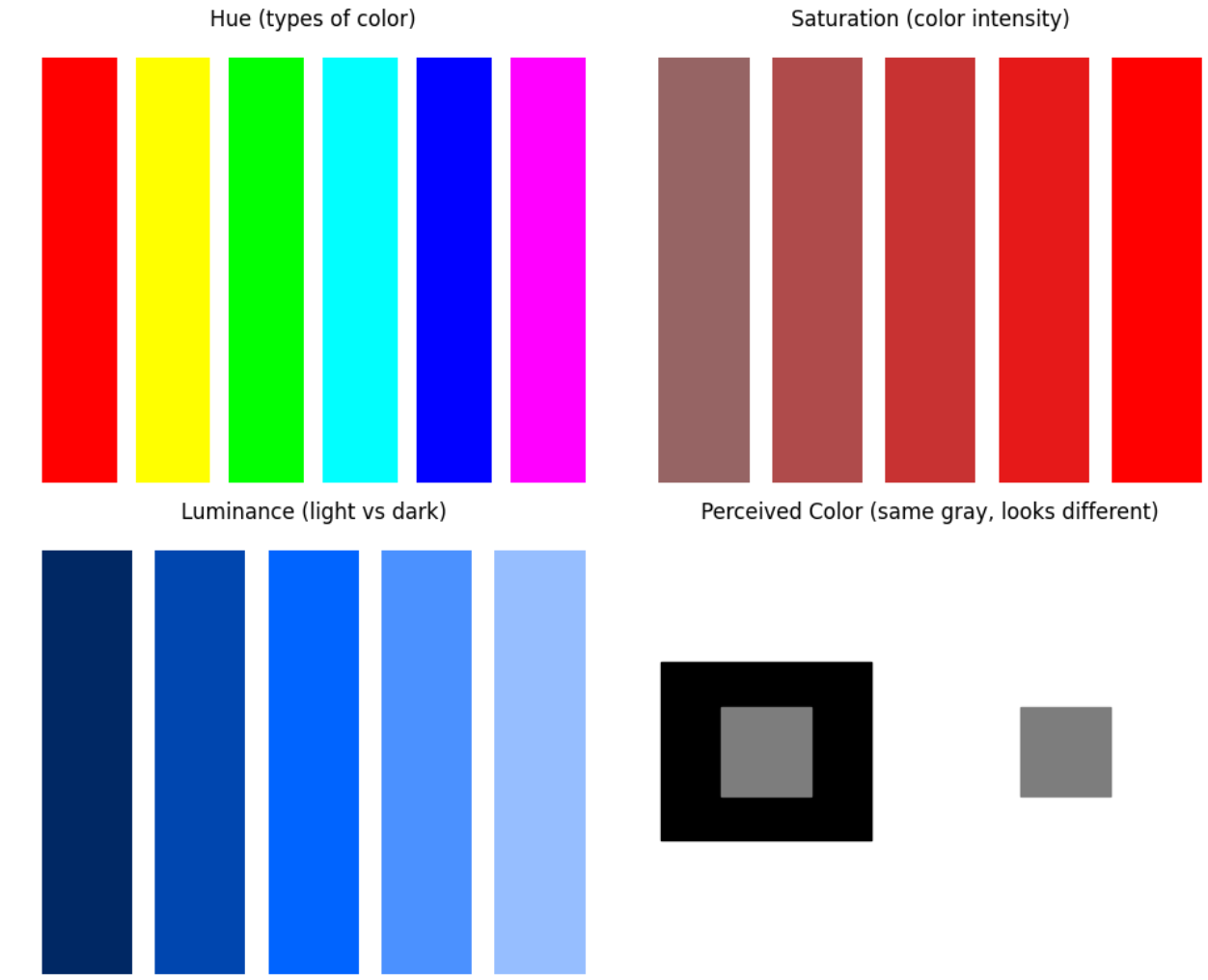
# Dark background with gray square
axes[1, 1].add_patch(plt.Rectangle((0.05, 0.3), 0.35, 0.4, color="black")
axes[1, 1].add_patch(plt.Rectangle((0.15, 0.4), 0.15, 0.2, color="gray"))

# Light background with gray square
axes[1, 1].add_patch(plt.Rectangle((0.55, 0.3), 0.35, 0.4, color="white")
```

```
axes[1, 1].add_patch(plt.Rectangle((0.65, 0.4), 0.15, 0.2, color="gray"))

axes[1, 1].set_title("Perceived Color (same gray, looks different)")

# ------------------------------
plt.tight_layout()
plt.show()
```



Task 1: Color and Perception

Write Python code that creates three bar charts showing the effect of:

Hue (different colors for each bar)

Saturation (same color, but vary saturation)

Luminance (same color, but vary brightness)

Use matplotlib or seaborn. Label each chart properly.

In [ ]:
```python
import matplotlib.pyplot as plt
import colorsys

categories = ['R', 'A', 'L', 'P', 'H']
values = [13, 26, 19, 29, 23]

def make_color(hue, lightness, saturation):
    return colorsys.hls_to_rgb(hue, lightness, saturation)

# 1. Different Hues (varying color type)
hue_colors = [make_color(h/5.0, 0.5, 0.8) for h in range(5)]

# 2. Varying Saturation (same red hue, changing color intensity)
sat_colors = [make_color(0.0, 0.5, s) for s in [0.2, 0.4, 0.6, 0.8, 1.0]]

# 3. Varying Luminance (same blue hue, changing brightness)
lum_colors = [make_color(0.6, l, 0.8) for l in [0.1, 0.3, 0.5, 0.7, 0.9]]

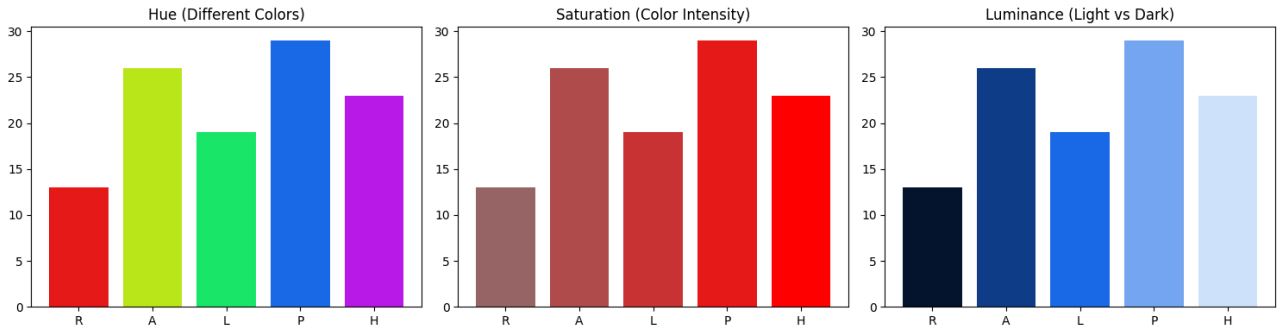
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].bar(categories, values, color=hue_colors)
axes[0].set_title('Hue (Different Colors)')

axes[1].bar(categories, values, color=sat_colors)
axes[1].set_title('Saturation (Color Intensity)')

axes[2].bar(categories, values, color=lum_colors)
axes[2].set_title('Luminance (Light vs Dark)')

plt.tight_layout()
plt.show()
```



Color Schemes and Types of Data

1. Color Schemes and Types of Data

Choosing the right color scheme depends on the type of data being visualized. The main types are:

Categorical data: Distinct groups or categories (e.g., gender, product type).

Sequential data: Ordered data where values progress from low to high (e.g., age, income, population).

Diverging data: Data with a meaningful midpoint, where deviation in both directions is important (e.g., profit vs. loss, above vs. below average).

The effectiveness of a visualization often depends on how well the color scheme matches the data type.

1. Categorical Color Schemes

Best for nominal (non-numeric) data.

Each category should have a distinct, easily distinguishable color.

Colors should not imply ordering (e.g., avoid gradients).

Example: Different regions of a country, or product categories.

1. Sequential Color Schemes

Used for ordered/numeric data that progresses from low to high.

A single hue with changes in luminance (light to dark) or saturation is often used.

Example: Population density, exam scores, rainfall.

Darker or more intense shades usually represent higher values.

1. Diverging Color Schemes

Used when data has a critical midpoint (e.g., zero, average, normal range).

Two contrasting hues show deviations in opposite directions.

Example: Positive vs. negative profit, temperature changes relative to average, political leaning.

Neutral or white/light color usually represents the midpoint.

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np

# Sample data for categorical example
categories = ["A", "B", "C", "D", "E"]
values = [10, 30, 20, 40, 25]
```

```
# Grid for sequential and diverging examples
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z_seq = np.sqrt(X**2 + Y**2)
Z_div = X

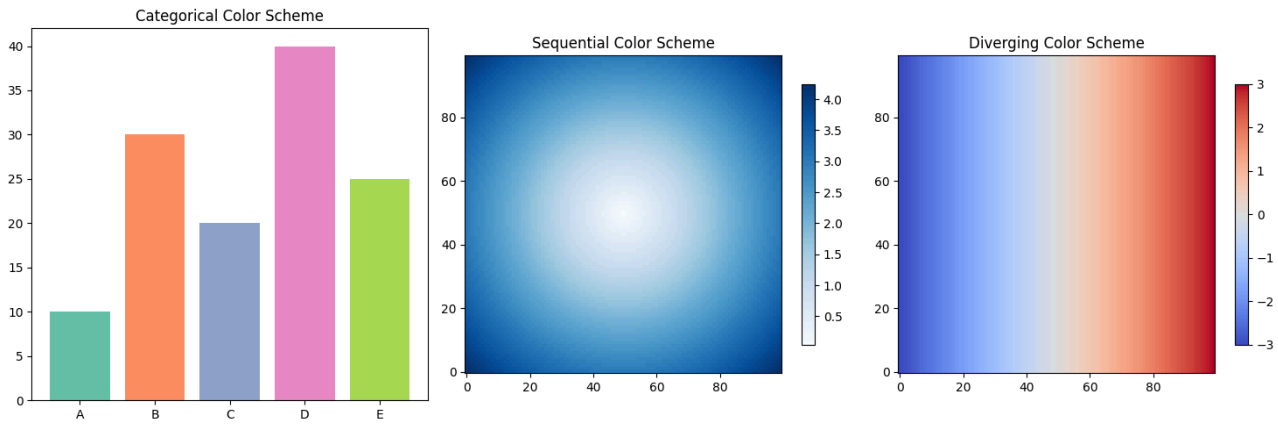
# Create figure
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# ------------------------------
# 1. Categorical Color Scheme
# ------------------------------
axes[0].bar(categories, values, color=plt.cm.Set2.colors[:len(categories)])
axes[0].set_title("Categorical Color Scheme")

# ------------------------------
# 2. Sequential Color Scheme
# ------------------------------
seq = axes[1].imshow(Z_seq, cmap="Blues", origin="lower")
axes[1].set_title("Sequential Color Scheme")
plt.colorbar(seq, ax=axes[1], shrink=0.7)

# ------------------------------
# 3. Diverging Color Scheme
# ------------------------------
div = axes[2].imshow(Z_div, cmap="coolwarm", origin="lower")
axes[2].set_title("Diverging Color Scheme")
plt.colorbar(div, ax=axes[2], shrink=0.7)

# Show results
plt.tight_layout()
plt.show()
```



Task 2: Color Schemes and Types of Data

Create three different plots using sample data to demonstrate:

Categorical Scheme (different categories with distinct colors)

Sequential Scheme (values increasing in a gradient from light to dark)

Diverging Scheme (centered around a midpoint, such as negative to positive values)

Use seaborn or matplotlib colormaps (Set2, Blues, RdBu).

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

# 1. Categorical Scheme (Distinct categories using Set2)
categories = ['North', 'South', 'East', 'West', 'Radius']
sales = [10, 25, 18, 30, 22]
sns.barplot(x=categories, y=sales, palette='Set2', ax=axes[0])
axes[0].set_title('Categorical Scheme (Set2)')

# 2. Sequential Scheme (Gradient from low to high using Blues)
grid_data = np.linspace(1, 100, 100).reshape(10, 10)
sns.heatmap(grid_data, cmap='Blues', ax=axes[1], cbar=True)
axes[1].set_title('Sequential Scheme (Blues)')

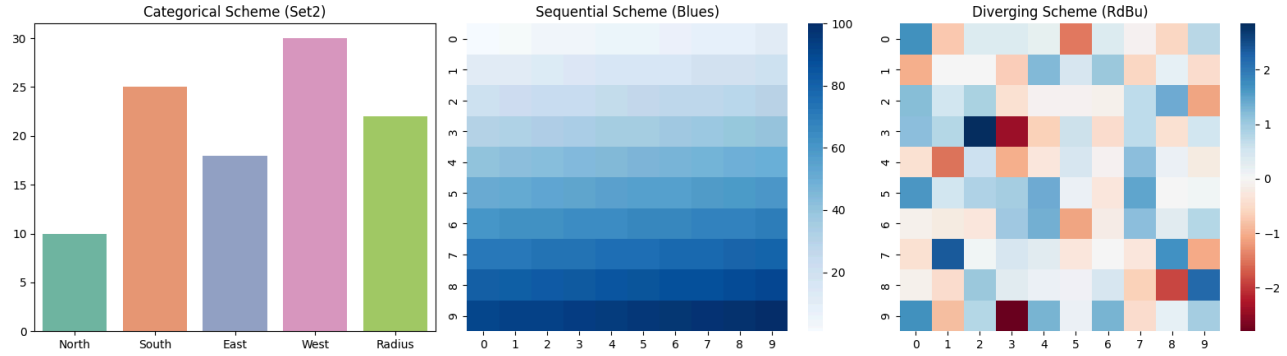
# 3. Diverging Scheme (Centered around zero using RdBu)
diverging_data = np.random.randn(10, 10)
sns.heatmap(diverging_data, cmap='RdBu', center=0, ax=axes[2], cbar=True)
axes[2].set_title('Diverging Scheme (RdBu)')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_3394/3928007661.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remov
ed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo
r the same effect.

  sns.barplot(x=categories, y=sales, palette='Set2', ax=axes[0])
```



Custom Color Using the HSL Color System

HSL Color System

The HSL model represents colors in terms of:

Hue (H): The type of color, represented as an angle on the color wheel (0–360°).

0° = Red, 120° = Green, 240° = Blue, etc.

Saturation (S): The intensity or purity of the color (0% = gray, 100% = vivid).

Luminance (L): The brightness of the color (0% = black, 50% = “normal” color, 100% = white).

Compared to RGB, the HSL model is more intuitive for humans, since we can directly control how colorful and how bright/dark a color looks.

Why use HSL in Data Visualization?

Customizability: Easily adjust brightness and vividness of the same hue.

Consistency: Keeps relationships across categories while maintaining visual balance.

Highlighting: By changing luminance or saturation, you can emphasize certain data points without changing the hue.

In [ ]:
```
import matplotlib.pyplot as plt
import colorsys
import numpy as np

# Function to convert HSL to RGB (Matplotlib uses RGB)
def hsl_to_rgb(h, s, l):
    return colorsys.hls_to_rgb(h, l, s)

# Create figure
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# ----------------------------
# 1. Varying Hue (different base colors)
# ----------------------------
hues = np.linspace(0, 1, 6, endpoint=False)  # 6 steps around the color w
colors_hue = [hsl_to_rgb(h, 1, 0.5) for h in hues]

axes[0].bar(range(len(hues)), [1]*len(hues), color=colors_hue)
axes[0].set_title("Varying Hue (0°–360°)")
```

```
axes[0].axis("off")

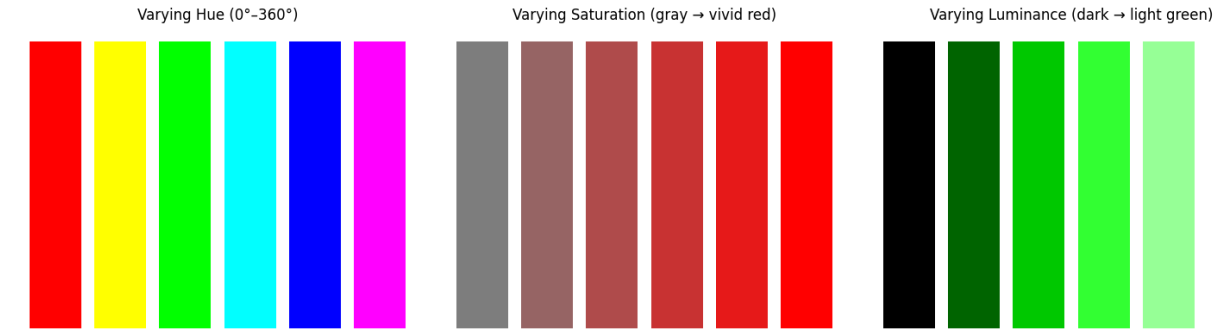
# ----------------------------
# 2. Varying Saturation (from grayish to vivid red)
# ----------------------------
saturations = np.linspace(0, 1, 6)
colors_sat = [hsl_to_rgb(0, s, 0.5) for s in saturations]  # Hue=0 → Red

axes[1].bar(range(len(saturations)), [1]*len(saturations), color=colors_s
axes[1].set_title("Varying Saturation (gray → vivid red)")
axes[1].axis("off")

# ----------------------------
# 3. Varying Luminance (dark → light green)
# ----------------------------
luminances = np.linspace(0, 1, 6)
colors_lum = [hsl_to_rgb(1/3, 1, l) for l in luminances]  # Hue=120° → Gr

axes[2].bar(range(len(luminances)), [1]*len(luminances), color=colors_lum
axes[2].set_title("Varying Luminance (dark → light green)")
axes[2].axis("off")

# ----------------------------
plt.tight_layout()
plt.show()
```



Task 3: Custom Color Using the HSL Color System

Write code that:

Generates 5 custom colors using the HSL system (different hues, fixed saturation/luminance).

Uses these custom colors to create a pie chart.

Prints the RGB values of each color to show the conversion.

Use colorsys or matplotlib.colors.hsv_to_rgb.

In [ ]:
```
import matplotlib.pyplot as plt
import colorsys
```

```python
import numpy as np

def hsl_to_rgb(h, s, l):
    return colorsys.hls_to_rgb(h, l, s)

hues = np.linspace(0, 1, 5, endpoint=False)
saturation = 0.75
luminance = 0.50

rgb_colors = []

# 1. Convert HSL to RGB and print values
print("HSL to RGB Conversions: ")
for h in hues:
    rgb = hsl_to_rgb(h, saturation, luminance)
    rgb_colors.append(rgb)

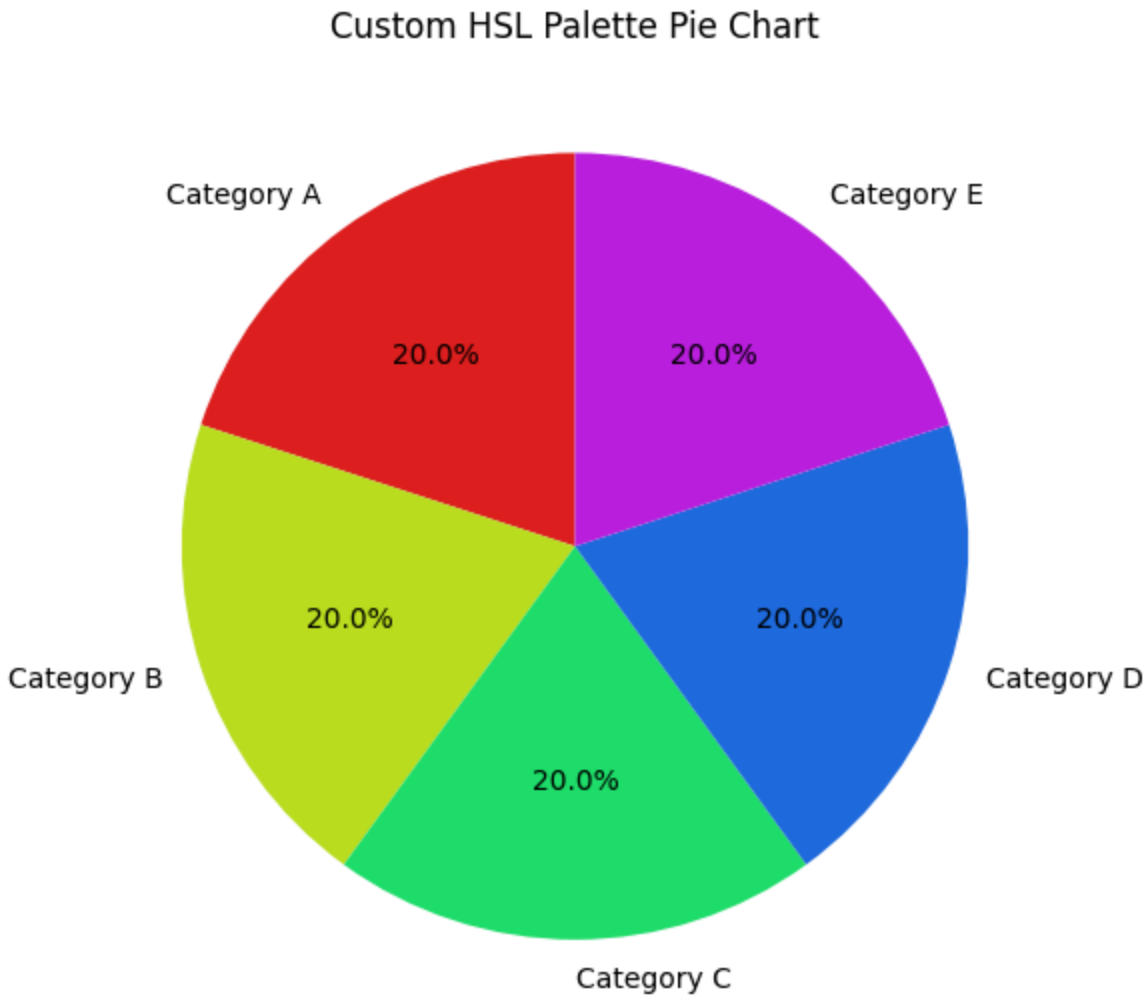

    hue_degrees = int(h * 360)
    rgb_scaled = tuple(round(c * 255) for c in rgb)
    print(f"Hue: {hue_degrees}° | RGB (0-1): {tuple(round(c, 2) for c in

# 2. Pie chart data using the custom HSL palette
labels = ['Category A', 'Category B', 'Category C', 'Category D', 'Catego
sizes = [20, 20, 20, 20, 20]

# 3. Create and plot pie chart
plt.figure(figsize=(6, 6))
plt.pie(sizes, labels=labels, colors=rgb_colors, autopct='%1.1f%%', start
plt.title("Custom HSL Palette Pie Chart")

plt.tight_layout()
plt.show()
```

```
HSL to RGB Conversions:
Hue: 0° | RGB (0-1): (0.88, np.float64(0.12), 0.12) | RGB (0-255): (223,
32, 32)
Hue: 72° | RGB (0-1): (np.float64(0.72), 0.88, 0.12) | RGB (0-255): (185,
223, 32)
Hue: 144° | RGB (0-1): (0.12, 0.88, np.float64(0.43)) | RGB (0-255): (32,
223, 108)
Hue: 216° | RGB (0-1): (0.12, np.float64(0.42), 0.88) | RGB (0-255): (32,
108, 223)
Hue: 288° | RGB (0-1): (np.float64(0.72), 0.12, 0.88) | RGB (0-255): (18
5, 32, 223)
```



Common Mistakes in the Use of Color in Data Visualization

Common Mistakes in the Use of Color in Data Visualization

Color is powerful for highlighting, grouping, and guiding attention, but when misused, it can confuse or mislead. Below are the most frequent mistakes:

1. Unnecessary Color

Using color where it does not add value.

Example: A bar chart where every bar is a different random color, even though the variable is a single category.

Correct approach: Use a single neutral color unless highlighting is needed.

1. Excessive Color

Using too many colors at once can overwhelm the reader.

Example: Pie chart with 20+ slices, each with a unique color, making it unreadable.

Correct approach: Limit the palette to what the eye can distinguish clearly (usually 6–8 distinct hues).

1. Insufficient Contrast

Colors that are too similar in shade make it hard to differentiate categories.

Example: Two shades of light blue side by side.

Correct approach: Ensure sufficient contrast, especially for small or adjacent elements.

1. Inconsistency Across Related Charts

Using different color schemes for the same categories across multiple charts causes confusion.

Example: “Product A” is blue in one chart and red in another.

Correct approach: Keep color assignments consistent across related visuals.

1. Neglecting Colorblindness

About 8% of men and 0.5% of women have some form of color vision deficiency.

Example: Using only red and green to distinguish categories.

Correct approach: Use colorblind-friendly palettes (e.g., viridis, cividis, or Color Universal Design guidelines).

1. Not Considering the Mode of Delivery

Visualizations may look different when printed in grayscale, projected, or viewed on different devices.

Example: A subtle gradient may disappear in black-and-white print.

Correct approach: Test visuals in the medium where they will be delivered.

In [ ]:
```
import matplotlib.pyplot as plt
```

```python
import seaborn as sns
import numpy as np

# Sample data
categories = ["A", "B", "C", "D"]
values = [10, 25, 17, 30]

fig, axs = plt.subplots(3, 2, figsize=(12, 10))

# 1. Unnecessary Color (Bad)
axs[0,0].bar(categories, values, color=["red", "blue", "green", "purple"])
axs[0,0].set_title("Bad: Unnecessary Colors")

# Good: One meaningful color
axs[0,1].bar(categories, values, color="steelblue")
axs[0,1].set_title("Good: Simple Color")

# 2. Excessive Color (Bad)
colors = sns.color_palette("hsv", 12)
axs[1,0].bar(range(12), np.random.randint(5,20,12), color=colors)
axs[1,0].set_title("Bad: Excessive Colors")

# Good: Limited palette
sns.barplot(x=categories, y=values, palette="Set2", ax=axs[1,1])
axs[1,1].set_title("Good: Limited Colors")

# 3. Insufficient Contrast (Bad)
axs[2,0].bar(categories, values, color="yellow")
axs[2,0].set_facecolor("white")
axs[2,0].set_title("Bad: Low Contrast")

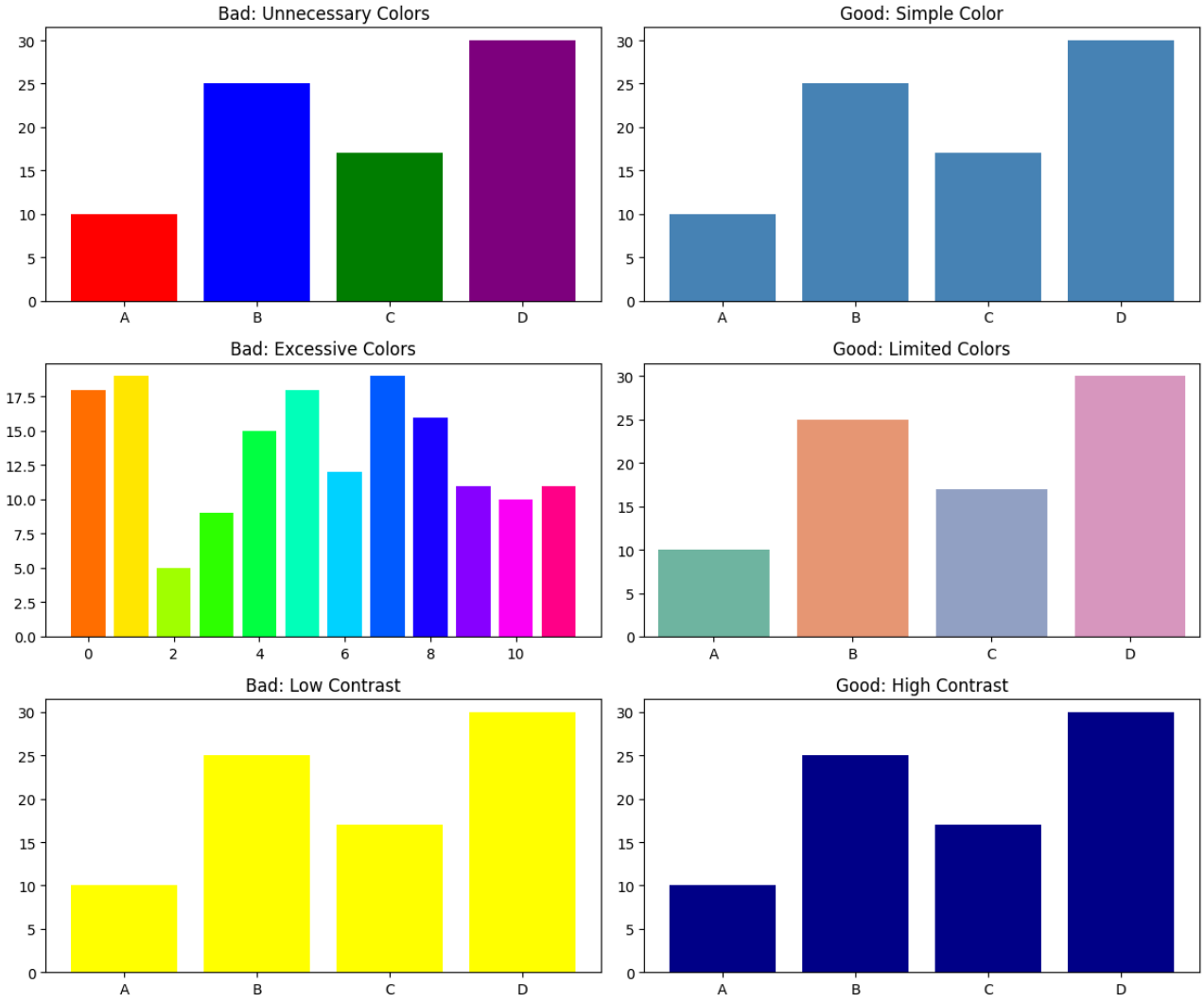
# Good: High Contrast
axs[2,1].bar(categories, values, color="darkblue")
axs[2,1].set_facecolor("white")
axs[2,1].set_title("Good: High Contrast")

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_3394/2400638606.py:25: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remov
ed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo
r the same effect.

  sns.barplot(x=categories, y=values, palette="Set2", ax=axs[1,1])
```



Task 4: Common Mistakes in the Use of Color in Data Visualization

Create two sets of bar charts:

1. Bad Examples

Use unnecessary colors (different random colors for each bar).

Use poor contrast (yellow on white background).

Use excessive colors (too many categories with different hues).

1. Good Examples

Use a single meaningful color.

Ensure high contrast.

Use a consistent limited palette.

Compare “bad” vs “good” side by side.

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Sample data
categories = ['A', 'B', 'C', 'D']
values = [10, 25, 17, 30]

fig, axes = plt.subplots(3, 2, figsize=(12, 10))

# 1. Unnecessary color vs single color
axes[0, 0].bar(categories, values, color=['red', 'blue', 'green', 'yellow
axes[0, 0].set_title('Bad: Unnecessary Colors')

axes[0, 1].bar(categories, values, color='steelblue')
axes[0, 1].set_title('Good: Simple Color')

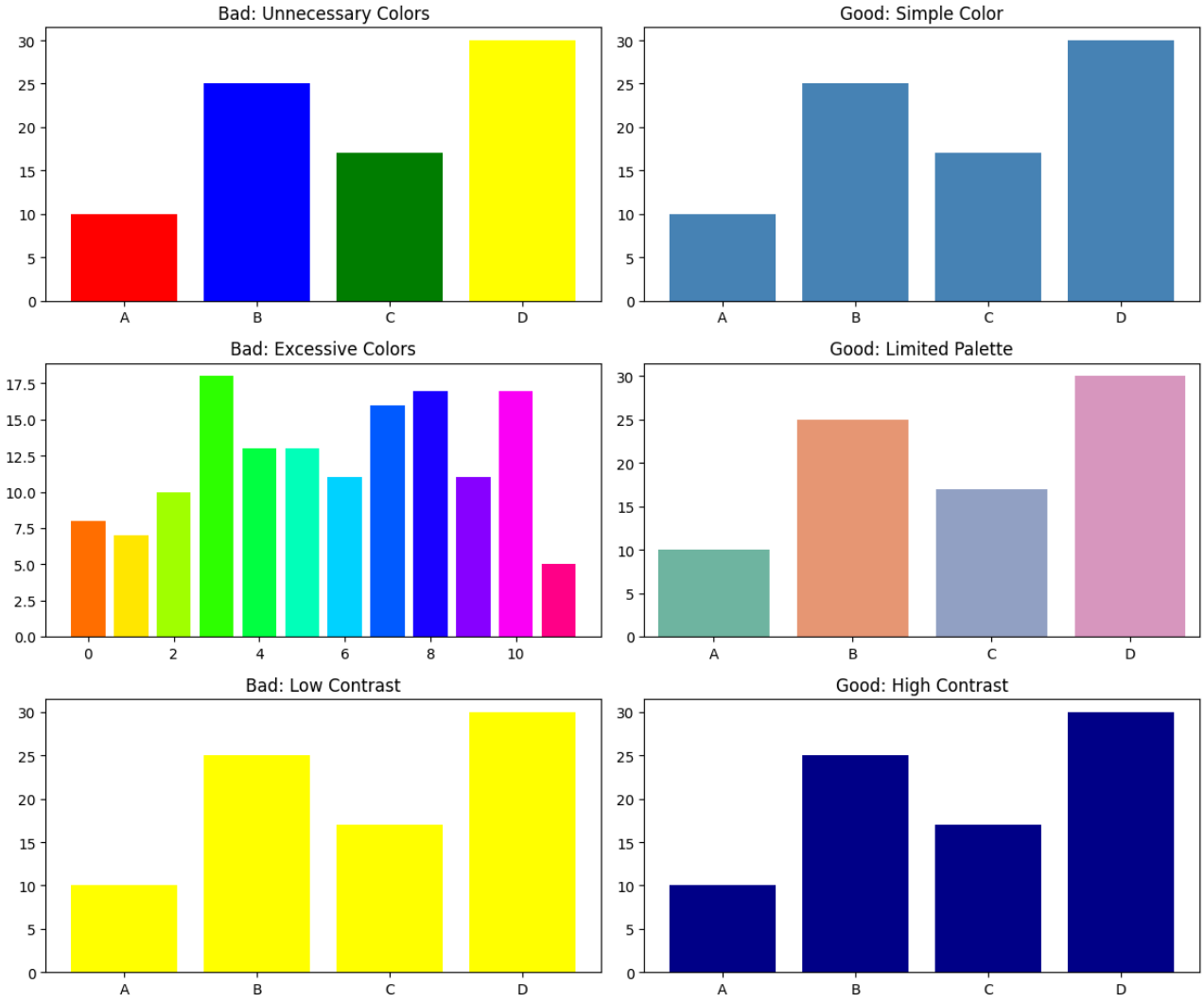
# 2. Excessive colors vs limited palette
many_vals = np.random.randint(5, 20, 12)
axes[1, 0].bar(range(12), many_vals, color=sns.color_palette('hsv', 12))
axes[1, 0].set_title('Bad: Excessive Colors')

sns.barplot(x=categories, y=values, hue=categories, palette='Set2', ax=ax
axes[1, 1].set_title('Good: Limited Palette')

# 3. Poor contrast vs high contrast
axes[2, 0].bar(categories, values, color='yellow')
axes[2, 0].set_facecolor('white')
axes[2, 0].set_title('Bad: Low Contrast')

axes[2, 1].bar(categories, values, color='darkblue')
axes[2, 1].set_facecolor('white')
axes[2, 1].set_title('Good: High Contrast')

plt.tight_layout()
plt.show()
```



---

5. Supplementary Activity:

Create a visualization project that demonstrates the purposeful use of color. Follow the steps below in one notebook/script.

1. Create a dataset

Example: 5 product categories with their sales values.

1. Color and Perception

Plot three bar charts: one showing hue differences, one showing saturation differences, and one showing luminance differences.

1. Color Schemes and Types of Data

Plot three more charts:

A categorical color scheme (each product = unique color).

A sequential color scheme (low → high values).

A diverging color scheme (below vs above average).

1. Custom Colors with HSL

Generate 5 custom HSL colors, convert them to RGB, and use them in a pie chart.

Print the RGB values of your chosen colors.

1. Common Mistakes and Corrections

Create one “bad” chart that uses too many random colors or poor contrast.

Create a corrected “good” version of the same chart with proper color use.

In [2]:
```
import colorsys
import os
import matplotlib.pyplot as plt
import pandas as pd


file_path = 'mobile_usage_behavioral_analysis.csv'


if os.path.exists(file_path):
    df = pd.read_csv(file_path)
    suppact = (
        df.groupby('Location')['Daily_Screen_Time_Hours'].mean().reset_in
    )
    categories = suppact['Location'].tolist()
    values = suppact['Daily_Screen_Time_Hours'].tolist()
else:
    categories = ['Chicago', 'Houston', 'Los Angeles', 'New York', 'Phoen
    values = [8.14, 7.45, 7.60, 7.40, 7.95]


hsl_colors = [
    (0.00, 1.0, 0.5),  # Red
    (0.08, 1.0, 0.5),  # Orange
    (0.16, 1.0, 0.5),  # Yellow
    (0.33, 1.0, 0.4),  # Green
    (0.60, 1.0, 0.5),  # Blue
]
```

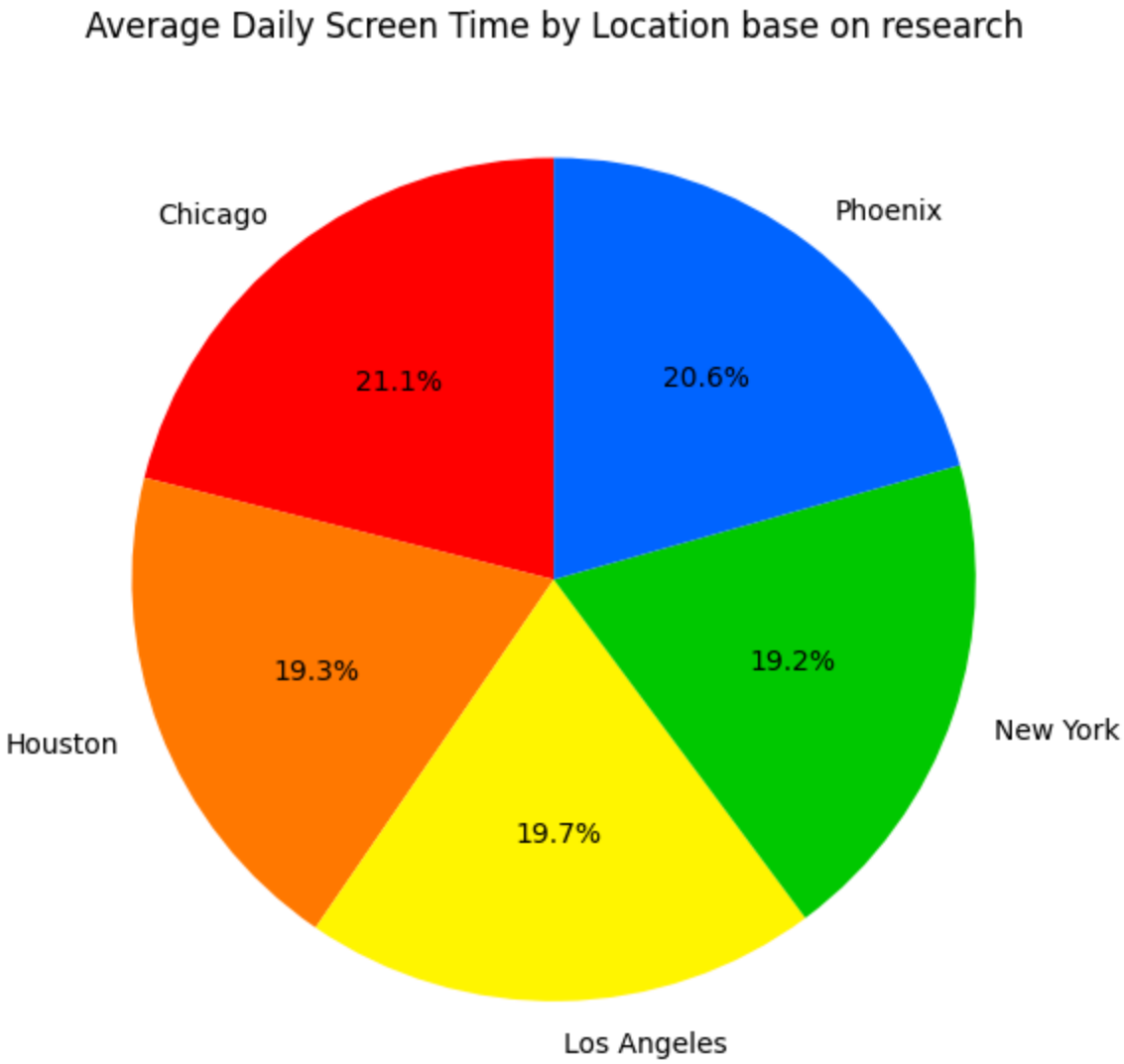
```python
# converts HSL to RGB
rgb_normalized = []
print("Converted RGB Values (0-255 format):")


for i, (h, s, l) in enumerate(hsl_colors):
    r, g, b = colorsys.hls_to_rgb(h, l, s)
    rgb_normalized.append((r, g, b))


    rgb_255 = (round(r * 255), round(g * 255), round(b * 255))
    print(f"{categories[i]}: RGB{rgb_255}")


# Pie Chart
plt.figure(figsize=(6, 6))
plt.pie(
    values,
    labels=categories,
    colors=rgb_normalized,
    autopct='%1.1f%%',
    startangle=90,
)
plt.title('Average Daily Screen Time by Location base on research')
plt.tight_layout()
plt.show()
```

```
Converted RGB Values (0-255 format):
Chicago: RGB(255, 0, 0)
Houston: RGB(255, 122, 0)
Los Angeles: RGB(255, 245, 0)
New York: RGB(4, 204, 0)
Phoenix: RGB(0, 102, 255)
```



6. Conclusion/Learnings/Analysis:

Working with real data showed me how color, brightness, and intensity completely changes how easy a chart is to read. Picking the right color setup—whether for separate groups, ranking high and low values, or comparing data to an average—makes patterns jump right out. Converting HSL colors to RGB gave me total freedom to build my own custom color palettes. In the end, using smart and simple color choices makes charts look clean, visually appealing, and easy for anyone to understand.